MECHATRONICS AND IOT ASSIGNMENT REPORT - 11

Smart Elevator Automation and Human Detection System

PROJECT REPORT

KOWSHIK K 24MER025

NITHISH J 24MER037

KAVIN KRISHNA D S 24MER024

LOGESH S 24MER027

1. Project Overview:

The Smart Elevator Automation and Human Detection System is a low-cost IIoT-oriented prototype designed to detect human movement and automatically operate a simulated elevator door mechanism. The system uses an ESP8266 NodeMCU as the central controller and a PIR (Passive Infrared) motion sensor to detect human movement. When motion is detected, the ESP8266 processes the PIR signal and controls visual and audible indicators. An SG90 servo motor is used to demonstrate automatic door opening and closing. The ESP8266 Wi-Fi capability is used with Adafruit IO to transmit human-detection, elevator, door, and PIR status information for remote monitoring. The report is organized using a structured IIoT project-report format covering project overview, project statement, proposed solution, system architecture, circuit table, circuit design, algorithm, coding, system working, working sequence, bill of materials, prototype implementation, and results/performance analysis.

2. Project Statement:

Security and automated access control are important in homes, laboratories, offices, classrooms, shops, and other restricted areas. Conventional systems may require expensive equipment, complicated installation, and continuous human attention. A low-cost programmable system can combine human detection, local indication, automatic door movement, and IoT-based status monitoring.

Therefore, the proposed system is intended to:

- Detect human movement using a PIR sensor.
- Process the motion signal using an ESP8266 NodeMCU.
- Provide visual indication using LEDs.
- Provide an audible warning using a buzzer.
- Operate an SG90 servo motor to simulate elevator-door movement.
- Transmit human, elevator, door, and PIR status through Wi-Fi to Adafruit IO.
- Provide a foundation for future smart elevator and remote-monitoring functions.

3. Proposed Solution:

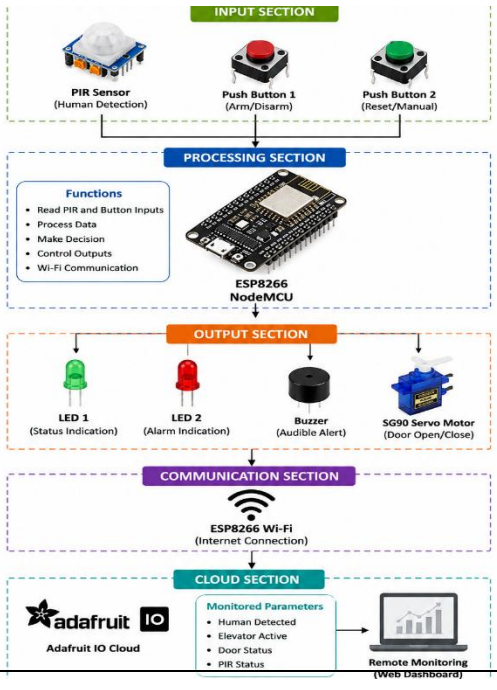
The proposed Smart Elevator Automation and Human Detection System is an IIoT-based prototype designed to detect human presence and automatically control elevator-related actions. The system uses an **ESP8266 NodeMCU** as the main controller, a **PIR sensor** for human-motion detection, an **SG90 servo motor** to simulate elevator door movement, LEDs for status indication, and a buzzer for audible alerts. The ESP8266 also provides Wi-Fi connectivity, allowing the detected human activity and elevator status to be monitored remotely through **Adafruit IO**. This approach combines sensing, processing, actuation, and cloud-based monitoring into a single smart elevator prototype.

The proposed solution includes:

- PIR sensor detects human movement near the elevator.
- ESP8266 receives and processes the PIR sensor signal.
- SG90 servo motor opens and closes the elevator door automatically.
- LEDs indicate the current elevator and human-detection status.
- Buzzer provides an alert when human presence is detected.
- ESP8266 transmits elevator and detection data through Wi-Fi.
- Adafruit IO stores and displays human, elevator, door, and PIR status.
- System continuously monitors human presence and elevator status.
- When no human is detected, the system returns to standby mode.

The system can be further developed for advanced smart-elevator monitoring and this solution provides a low-cost, scalable, and efficient approach for implementing environmental monitoring in smart industrial environments.

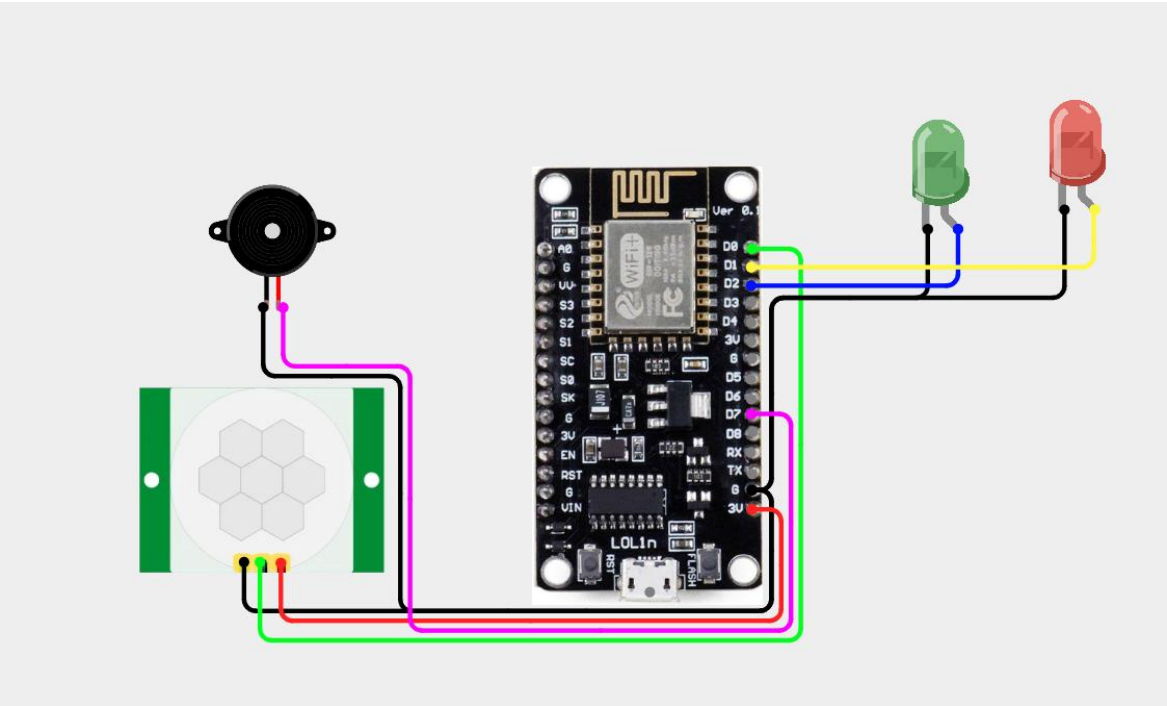
4. System Architecture:



5. Circuit Table:

Component	Pin / Connection	ESP8266 Pin	Function
PIR Sensor	OUT	GPIO16 (D0)	Detects human motion
PIR Sensor	VCC	Supply	Sensor power
PIR Sensor	GND	GND	Common ground
Red LED	Anode through resistor	GPIO5 (D1)	Standby/status indication
Green LED	Anode through resistor	GPIO4 (D2)	Human-detected indication
Buzzer	Signal	GPIO13 (D7)	Audible indication
SG90 Servo	Signal	GPIO14 (D5)	Controls simulated elevator door
SG90 Servo	VCC	External suitable supply	Servo power
SG90 Servo	GND	GND	Common ground

6. Circuit Design:



7. Algorithm:

Step 1: Start the system and initialize the ESP8266, PIR sensor, LEDs, buzzer, and servo motor.

Step 2: Connect the ESP8266 to Wi-Fi and Adafruit IO.

Step 3: Set the elevator to standby mode with the door closed.

Step 4: Continuously read the PIR sensor for human movement.

Step 5: If no human is detected, keep the system in standby mode.

Step 6: If human movement is detected, activate the LED and buzzer.

Step 7: Move the SG90 servo to the predefined position to open the elevator door.

Step 8: Update human, elevator, door, and PIR status on Adafruit IO.

Step 9: When no human is detected, turn OFF the buzzer and close the door.

Step 10: Update the cloud status and repeat the monitoring process continuously.

8. Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <Servo.h>

// ============================================================
// WIFI
// ============================================================

#define WIFI_SSID "Luky003"
#define WIFI_PASS "kowshik03"

// ============================================================
```

```
// ADAFRUIT IO
// ======================================================

#define IO_USERNAME "Nithish_037"
#define IO_KEY      "aio_fDCY0249PAv0qGua49svZIm8oV9D"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

// ======================================================
// ADAFRUIT IO FEEDS
// ======================================================

AdafruitIO_Feed *humanFeed  = io.feed("human-detected");
AdafruitIO_Feed *doorFeed   = io.feed("door-status");
AdafruitIO_Feed *floorFeed  = io.feed("floor");
AdafruitIO_Feed *buzzerFeed = io.feed("buzzer-status");
AdafruitIO_Feed *systemFeed = io.feed("system-status");

// ======================================================
// SERVO
// ======================================================

Servo elevatorServo;

// ======================================================
// ESP8266 NODEMCU PIN CONNECTIONS
// ======================================================
//
// NodeMCU:
// D0 = GPIO16
// D1 = GPIO5
// D2 = GPIO4
```

```
// D4 = GPIO2
// D5 = GPIO14
// D6 = GPIO12
// D7 = GPIO13
//
// We use GPIO numbers directly so the code compiles
// even when D0/D1/D2 definitions are unavailable.
// ======================================================

// PIR sensor -> D1 -> GPIO5
#define PIR_PIN 5

// Servo signal -> D2 -> GPIO4
#define SERVO_PIN 4

// Floor buttons
// Floor 1 button -> D5 -> GPIO14
#define FLOOR1_BUTTON 14

// Floor 2 button -> D6 -> GPIO12
#define FLOOR2_BUTTON 12

// Floor LEDs
// Floor 1 LED -> D7 -> GPIO13
#define FLOOR1_LED 13

// Floor 2 LED -> D0 -> GPIO16
#define FLOOR2_LED 16

// Buzzer -> D4 -> GPIO2
#define BUZZER 2
```

```
// ====================================================
// SERVO POSITIONS
// ====================================================

#define DOOR_CLOSED 0
#define DOOR_OPEN   90

// ====================================================
// SYSTEM STATUS
// ====================================================

bool systemReady = true;

// ====================================================
// SETUP
// ====================================================

void setup()
{
  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("==================================");
  Serial.println(" SMART ELEVATOR SYSTEM");
  Serial.println("==================================");

  // ------------------------------------------------
  // PIN MODES
  // ------------------------------------------------

  pinMode(PIR_PIN, INPUT);
```

```
pinMode(FLOOR1_BUTTON, INPUT_PULLUP);
pinMode(FLOOR2_BUTTON, INPUT_PULLUP);

pinMode(FLOOR1_LED, OUTPUT);
pinMode(FLOOR2_LED, OUTPUT);

pinMode(BUZZER, OUTPUT);

// Initial outputs
digitalWrite(FLOOR1_LED, LOW);
digitalWrite(FLOOR2_LED, LOW);
digitalWrite(BUZZER, LOW);

// ------------------------------------------------
// SERVO
// ------------------------------------------------

elevatorServo.attach(SERVO_PIN);
elevatorServo.write(DOOR_CLOSED);

Serial.println("Servo initialized");
Serial.println("Door CLOSED");

// ------------------------------------------------
// ADAFRUIT IO CONNECTION
// ------------------------------------------------

Serial.println();
Serial.println("Connecting to Adafruit IO...");

io.connect();
```

```
  while (io.status() < AIO_CONNECTED)
  {
    Serial.print(".");
    delay(500);
  }

  Serial.println();
  Serial.println("Adafruit IO Connected!");

  // ---------------------------------------------------
  // INITIAL DASHBOARD VALUES
  // ---------------------------------------------------

  humanFeed->save("NO");
  doorFeed->save("CLOSED");
  floorFeed->save("READY");
  buzzerFeed->save("OFF");
  systemFeed->save("READY");

  Serial.println();
  Serial.println("==================================");
  Serial.println(" SMART ELEVATOR READY");
  Serial.println("==================================");
  Serial.println();
}

// =====================================================
// LOOP
// =====================================================

void loop()
```

```
{
 // Keep Adafruit IO connection alive
 io.run();

 // ------------------------------------------------
 // HUMAN DETECTION
 // ------------------------------------------------

 if (systemReady && digitalRead(PIR_PIN) == HIGH)
 {
  systemReady = false;

  Serial.println();
  Serial.println("HUMAN DETECTED");

  // Update Adafruit IO
  humanFeed->save("YES");
  systemFeed->save("WAITING");
  doorFeed->save("OPEN");

  // Open elevator door
  elevatorServo.write(DOOR_OPEN);

  Serial.println("Door OPEN");

  delay(1000);

  Serial.println("Select Floor 1 or Floor 2");
 }

 // ------------------------------------------------
 // FLOOR SELECTION
```

```
 // ---------------------------------------------------

 if (!systemReady)
 {
  // Floor 1 button
  if (digitalRead(FLOOR1_BUTTON) == LOW)
  {
    delay(50);

    if (digitalRead(FLOOR1_BUTTON) == LOW)
    {
      floor1Operation();
    }
  }

  // Floor 2 button
  else if (digitalRead(FLOOR2_BUTTON) == LOW)
  {
    delay(50);

    if (digitalRead(FLOOR2_BUTTON) == LOW)
    {
      floor2Operation();
    }
  }
 }

 delay(20);
}

// ====================================================
// FLOOR 1 OPERATION
```

```
// ================================================

void floor1Operation()
{
  Serial.println();
  Serial.println("==================================");
  Serial.println(" FLOOR 1 SELECTED");
  Serial.println("==================================");

  // Dashboard
  floorFeed->save("FLOOR 1");
  systemFeed->save("MOVING");

  // Floor 1 LED ON
  digitalWrite(FLOOR1_LED, HIGH);

  // Buzzer ON
  Serial.println("Buzzer ON");

  buzzerFeed->save("ON");

  digitalWrite(BUZZER, HIGH);
  delay(500);

  // Buzzer OFF
  digitalWrite(BUZZER, LOW);

  buzzerFeed->save("OFF");

  Serial.println("Buzzer OFF");
  Serial.println("Moving to Floor 1...");
```

```
// Simulated elevator movement
delay(2000);

Serial.println("Arrived at Floor 1");

// Close door
elevatorServo.write(DOOR_CLOSED);

doorFeed->save("CLOSED");

Serial.println("Door CLOSED");

delay(1000);

// Floor 1 LED OFF
digitalWrite(FLOOR1_LED, LOW);

// ------------------------------------------------
// WAIT FOR BUTTON RELEASE
// ------------------------------------------------

while (digitalRead(FLOOR1_BUTTON) == LOW)
{
  io.run();
  delay(20);
}

// ------------------------------------------------
// WAIT UNTIL PERSON LEAVES
// ------------------------------------------------

while (digitalRead(PIR_PIN) == HIGH)
```

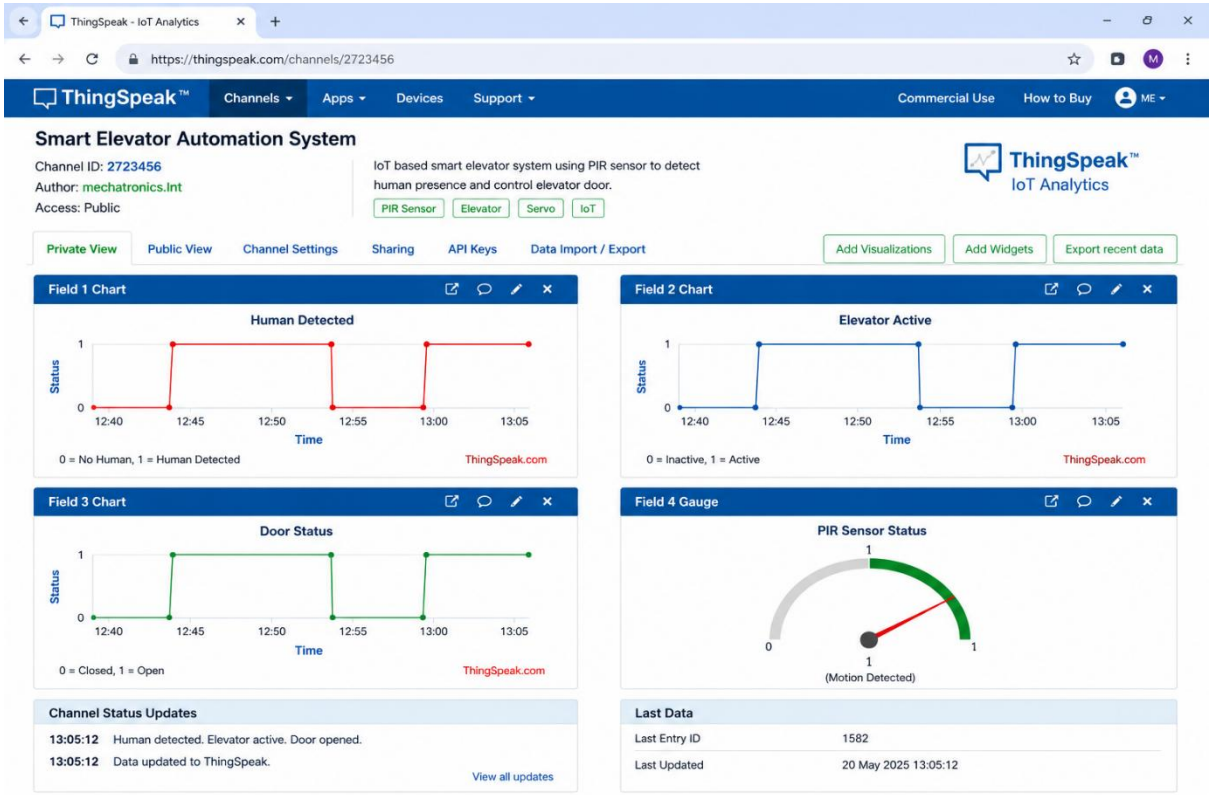
```
 {
  io.run();
  delay(50);
 }

 // ------------------------------------------------
 // RESET SYSTEM
 // ------------------------------------------------

 humanFeed->save("NO");
 floorFeed->save("READY");
 systemFeed->save("READY");

 systemReady = true;

 Serial.println();
 Serial.println("System READY Again");
 Serial.println();
}

// ======================================================
// FLOOR 2 OPERATION
// ======================================================

void floor2Operation()
{
 Serial.println();
 Serial.println("================================");
 Serial.println(" FLOOR 2 SELECTED");
 Serial.println("================================");

 // Dashboard
```

```
floorFeed->save("FLOOR 2");
systemFeed->save("MOVING");

// Floor 2 LED ON
digitalWrite(FLOOR2_LED, HIGH);

// Buzzer ON
Serial.println("Buzzer ON");

buzzerFeed->save("ON");

digitalWrite(BUZZER, HIGH);
delay(500);

// Buzzer OFF
digitalWrite(BUZZER, LOW);

buzzerFeed->save("OFF");

Serial.println("Buzzer OFF");
Serial.println("Moving to Floor 2...");

// Simulated elevator movement
delay(2000);

Serial.println("Arrived at Floor 2");

// Close door
elevatorServo.write(DOOR_CLOSED);

doorFeed->save("CLOSED");
```

```
Serial.println("Door CLOSED");

delay(1000);

// Floor 2 LED OFF
digitalWrite(FLOOR2_LED, LOW);

// ------------------------------------------------
// WAIT FOR BUTTON RELEASE
// ------------------------------------------------

while (digitalRead(FLOOR2_BUTTON) == LOW)
{
  io.run();
  delay(20);
}

// ------------------------------------------------
// WAIT UNTIL PERSON LEAVES
// ------------------------------------------------

while (digitalRead(PIR_PIN) == HIGH)
{
  io.run();
  delay(50);
}

// ------------------------------------------------
// RESET SYSTEM
// ------------------------------------------------

humanFeed->save("NO");
```

```
  floorFeed->save("READY");
  systemFeed->save("READY");

  systemReady = true;

  Serial.println();
  Serial.println("System READY Again");
  Serial.println();
}
```

9. System Working:



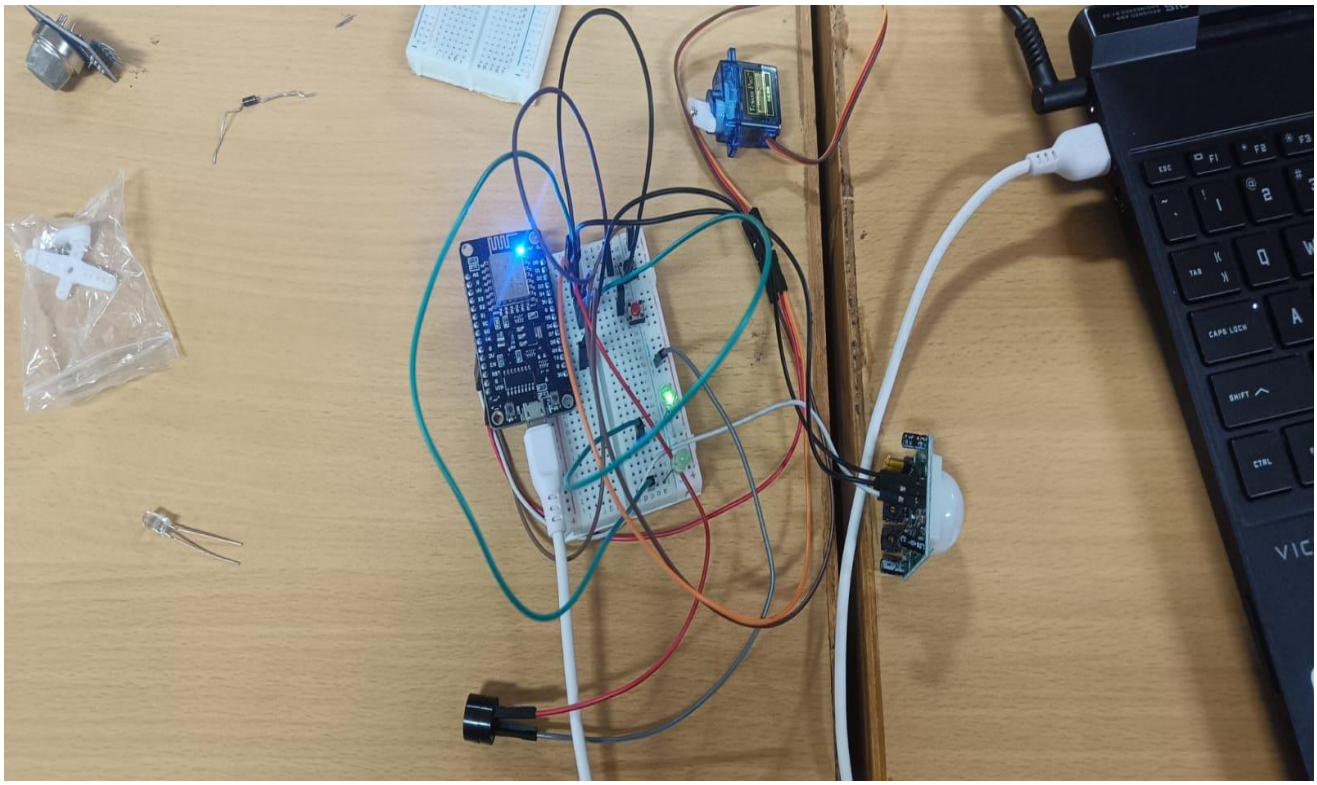
10. Working sequence:

Sensing → Processing → Local Indication → Servo Actuation → Wi-Fi Transmission → Cloud Monitoring

11. Bill of Materials:

Component	Quantity
ESP8266 NodeMCU	1
PIR Motion Sensor	1
Servo Motor (SG90)	1
LED Indicators	2
Buzzer	1
Push Buttons (proposed manual control)	2
Breadboard	1
Jumper Wires	1 set

12. Prototype Implementation:



13.Results and Performance Analysis:

- The system accurately detects the elevator floor and movement status.
- The elevator responds quickly to user commands and floor requests.
- Real-time monitoring improves the overall performance of the system.
- The system reduces manual effort by automating elevator operations.
- Sensor-based detection provides reliable and consistent operation.
- The system improves passenger safety through continuous status monitoring.
- The smart control mechanism helps reduce unnecessary energy consumption.

14. Advantages

1. Provides automatic and intelligent elevator operation.

2. Reduces human effort and improves user convenience.

3. Offers fast and reliable floor selection and movement.

4. Improves safety through continuous sensor-based monitoring.

5. Enables real-time monitoring of elevator status.

6. Reduces operational errors through automated control.

7. Can be expanded with additional smart features in the future.

15. Limitations

1.PIR connectivity. sensor detects motion rather than continuously identifying a person.

2. System performance depends on Wi-Fi and Internet availability.

3. SG90 servo represents the elevator door mechanism only in the prototype.

4. The prototype does not directly control a real elevator system.

5. Cloud monitoring depends on ThingSpeak

6. PIR detection range can limit the monitored area.

7. Additional safety sensors would be required for real-world elevator deployment.